

ENGINEERING BRIEF · AGENT B

Roomio — Furniture Detection Pipeline

A parallel work-stream for a second Claude instance: build a local, open-source furniture detection + color pipeline that takes a room photo and proposes furniture (category + color) for the user to confirm against Roomio's existing archetype corpus. Runs in its own folder, coordinates with the main app via shared files.

PARALLEL AGENT

LOCAL / OPEN-SOURCE

OLLAMA + YOLO

Repo: `~/Desktop/personal/roomio` · **Your folder:** `/detection-pipeline`

Coordination: `/shared/*.json` · **Joint log:** `roomio.txt` (repo root)

Date: June 29, 2026 · **Companion to:** Room Designer MVP Requirements v2

. Contents

1	Your Mission & Boundaries	03
2	The Product Goal (what detection is for)	03
3	Repo Layout & Isolation Rules	04
4	The Detection Pipeline (3 stages)	05
5	Recommended Open-Source Stack	07
6	Coordination Protocol (shared files)	08
7	The roomio.txt Communication Contract	10
8	Functional Requirements & Acceptance	11
9	Build Milestones	12

1. Your Mission & Boundaries

You are **Agent B**. A separate Claude instance ("Agent A") is building the main Roomio room-designer front-end in the same repo. Your job is the **furniture detection pipeline**: given a room photo, detect furniture, classify each into one of Roomio's existing archetype categories, extract a dominant color, and emit a structured proposal the front-end shows as confirmable dropdowns. You do *not* touch the front-end; you produce a clean service + a shared contract Agent A consumes.

Hard boundaries

- All your code lives in `/detection-pipeline`. Do not edit files outside it except the two shared channels in §6–7.
- Never modify Agent A's front-end source. If you need something from them, request it via `roomio.txt` (§7).
- Everything runs **locally and open-source** — no paid APIs, no cloud inference. Ollama for the vision model; Python for detection/color.
- Detection is a **suggestion layer only**. Wrong output must degrade to "user picks manually," never a crash or a blocked flow.

2. The Product Goal

Roomio already has a working "Furnish the room" step with an archetype corpus: Sofas (3-Seater, Loveseat, L-Shaped Sectional, Recliner), Beds (Single, Queen, King), Tables (Coffee, Dining, Round Dining, Side), Chairs (Dining, Armchair, Office, Accent), Storage (Wardrobe, Bookcase, TV Unit, Dresser), Decor (Rug, Floor Lamp, Potted Plant), and a Placeholder Box.

The detection pipeline's purpose: a user uploads/scans a room photo, and the system **recognizes the furniture from this existing corpus** and pre-fills the picker. The user is shown a dropdown per detected item ("We think this is a *3-Seater Sofa*, color *sage green* — correct?"). If it makes sense, the user accepts and it's added to the design; if not, they change the dropdown. Confirmation is always required — nothing is auto-added.

KEY CONSTRAINT

Detection output must map onto the *existing* archetype categories, not invent new ones. The corpus is the closed label set. Your classifier's job is "which of these known archetypes is this most likely to be," plus a confidence and a color.

3. Repo Layout & Isolation Rules

Create this structure under the existing repo so both agents run without collision:

```
# ~/Desktop/personal/roomio
roomio/
├─ (Agent A's existing front-end ... DO NOT TOUCH)
├─ roomio.txt           # JOINT requirements + comms log (both agents)
├─ shared/              # the contract surface between agents
│   ├── detection_schema.json # the agreed output shape (you define, A consumes)
│   ├── archetypes.json      # the closed label set (mirror of A's corpus)
│   ├── requests/            # A drops images here for you to process
│   └─ results/              # you drop detection results here for A
└─ detection-pipeline/    # YOUR sandbox – all your code lives here
    ├── README.md
    ├── requirements.txt
    ├── detect.py           # stage 1: bounding boxes
    ├── classify.py          # stage 2: map to archetype
    ├── color.py            # stage 3: dominant color
    ├── pipeline.py         # orchestrates 1-2-3, writes results/
    ├── watcher.py          # polls shared/requests/, runs pipeline
    └─ tests/
```

ISOLATION RULES (NON-NEGOTIABLE)

1. Write only inside `/detection-pipeline` and the `/shared` channels.
2. Treat `/shared/detection_schema.json` as a published API — if you must change its shape, announce it in `roomio.txt` first and wait for Agent A to acknowledge.
3. Use your own Python venv inside the folder; never install global packages that could affect Agent A's Node environment.
4. Your service communicates with the front-end through files (§6), not by importing each other's code.

4. The Detection Pipeline (3 stages)

Three decoupled stages so each can be swapped or upgraded independently. Input: a room photo. Output: a JSON proposal list.

Stage 1 — Detect (bounding boxes)

Find furniture regions in the photo. Two viable paths; support the fast one first, the flexible one as fallback:

- **Fast/deterministic:** a YOLO detector fine-tuned on the **HomeObjects-3K** dataset, whose classes (bed, sofa, chair, table, lamp, tv, wardrobe, window, door, potted plant, etc.) map almost 1:1 onto Roomio's corpus. Trains with one command; runs in milliseconds.
- **Flexible/zero-train:** a local vision-language model via Ollama that returns bounding boxes as JSON (see §5). Slower but needs no training and handles odd items.

Stage 2 — Classify (map to archetype)

For each detected region, decide which *Roomio archetype* it is. The detector gives a coarse class ("sofa"); you must refine to the corpus ("3-Seater Sofa" vs "Loveseat" vs "L-Shaped Sectional"). Use the VLM with a constrained prompt that lists the allowed archetypes and asks for the single best match + confidence. Always return the closed-set label, never free text.

Stage 3 — Color (dominant color of the region)

Crop the bounding box, run **k-means dominant-color extraction** (OpenCV + scikit-learn), discard near-white/near-black background clusters below a coverage threshold, convert the top centroid to a named color via nearest-match in a palette. Return both the hex and a human name ("sage green").

```
# pipeline.py - sketch
def process(image_path):
    boxes = detect(image_path)          # stage 1
    proposals = []
    for b in boxes:
        crop = crop_region(image_path, b.bbox)
        archetype, conf = classify(crop)  # stage 2 (closed-set)
        hex_, name = dominant_color(crop) # stage 3
        proposals.append({
            "detected_label": b.label,
            "archetype_id": archetype,    # MUST be in archetypes.json
            "confidence": conf,
            "color_hex": hex_, "color_name": name,
            "bbox": b.bbox
        })
    return proposals
```

5. Recommended Open-Source Stack

Stage	Tool	Why
Vision model (classify + optional boxes)	Qwen2.5-VL via Ollama (<code>qwen2.5vl:7b</code>)	Returns bounding boxes and stable JSON coordinates + attributes; strong at recognizing common objects and producing structured output — ideal for closed-set classification with a constrained prompt.
Lightweight fallback	moondream2 via Ollama (~1.9B)	Only practical choice under ~4 GB VRAM; fast for a first test. Weaker on complex scenes — use to validate plumbing.
General alt	llama3.2-vision 11B / LLaVA 1.6 7B	llama3.2-vision is a strong general local VLM (8–16 GB VRAM); LLaVA is the most documented if you need examples.
Fast detector (optional)	YOLO + HomeObjects-3K	One-command training; class list matches the corpus; ms-level inference for deterministic boxes.
Color	OpenCV + scikit-learn k-means	Standard dominant-color extraction; sample pixels in the crop, cluster in RGB/LAB, drop low-coverage near-white/black clusters, name via nearest palette match.
Runtime	Python 3 venv in <code>/detection-pipeline</code>	Isolated from Agent A's Node env.

```
# quickest path to a working VLM call (Ollama Python lib)
import ollama
r = ollama.chat(model="qwen2.5vl:7b", messages=[{
    "role": "user",
    "content": "List furniture in this image. For each, give a bounding box "
               "and pick ONE label from: [3-seater sofa, loveseat, l-shaped "
               "sectional, recliner, single bed, queen bed, king bed, coffee "
               "table, dining table, round dining table, side table, dining "
               "chair, armchair, office chair, accent chair, wardrobe, "
               "bookcase, tv unit, dresser, rug, floor lamp, potted plant]. "
               "Return ONLY JSON: [{label, bbox:[x,y,w,h], confidence}].",
    "images": ["room.jpg"]
}])
print(r["message"]["content"]) # parse, then run color stage per box
```

LICENSING WATCH-OUT

Ultralytics YOLO is commonly **AGPL-3.0**, which has network-use obligations for a hosted product. If Roomio will be hosted commercially, confirm a compatible detector or rely on the VLM path. Note this decision in `roomio.txt`. Verify each model's license before committing.

6. Coordination Protocol (shared files)

The two agents never call each other's code. They pass messages through files in `/shared`. This keeps the repo collision-free and lets each agent run independently.

6.1 The published contract: `detection_schema.json`

You author this; Agent A reads it to build the confirmation UI. Keep it stable. Example:

```
{
  "version": "1.0",
  "request": { "image_path": "shared/requests/<id>.jpg", "request_id": "<id>" },
  "result": {
    "request_id": "<id>",
    "status": "ok | error",
    "proposals": [
      {
        "archetype_id": "sofa_3_seater",    # closed set, from archetypes.json
        "display_label": "3-Seater Sofa",
        "confidence": 0.0,
        "color_hex": "#8a9a7b",
        "color_name": "sage green",
        "bbox": [x, y, w, h]
      }
    ],
    "generated_by": "agent_b",
    "model": "qwen2.5vl:7b"
  }
}
```

6.2 The runtime flow (request → result)

1. Agent A writes a photo to `shared/requests/<id>.jpg` and a tiny `<id>.request.json`.
2. Your `watcher.py` polls `shared/requests/`, runs the pipeline, writes `shared/results/<id>.result.json` matching the schema.
3. Agent A polls `shared/results/`, renders the confirmation dropdowns from the proposals.
4. Use an atomic write (write to `.tmp`, then rename) so A never reads a half-written file.

WHY FILES, NOT A SHARED IMPORT

File-based messaging means either agent can be restarted, run at its own pace, or be developed independently without breaking the other. It also gives you a natural audit trail of every request/result during the build.

6.3 `archetypes.json` (the closed label set)

Mirror Agent A's corpus here as the single source of truth for valid `archetype_id` values. If A adds an archetype, they update this file and note it in `roomio.txt`; you then support the new id. Never emit an `archetype_id` not present here — fall back to `placeholder_box`.

7. The roomio.txt Communication Contract

`roomio.txt` at the repo root is the **shared requirements + communication log** for both agents (and the human). You must (a) read it at the start of every work cycle, and (b) write to it whenever you make a decision, change the contract, or need something from Agent A.

Rules for writing to roomio.txt

- **Append, never overwrite.** It is a running log shared with Agent A and the human.
- Prefix every entry with a timestamp and your identity: `[YYYY-MM-DD HH:MM] [AGENT-B]`.
- State your **intent** before big changes ("about to change schema field X").
- Log any **cross-agent dependency** as an explicit request to Agent A.
- Record decisions a human or Agent A would need to know (model chosen, license note, schema version bump).

Entry format

```
# roomio.txt
[2026-06-29 22:10] [AGENT-B] INTENT: scaffolding /detection-pipeline,
  Python venv, will not touch front-end. Schema v1.0 published to
  shared/detection_schema.json – please confirm fields work for your UI.
[2026-06-29 22:35] [AGENT-B] REQUEST -> AGENT-A: please write your
  archetype corpus to shared/archetypes.json so my classifier uses
  your exact ids. Using placeholder list until then.
[2026-06-29 23:02] [AGENT-B] DECISION: using qwen2.5vl:7b via Ollama for
  classify+boxes; moondream2 fallback. YOLO path deferred (AGPL note).
[2026-06-29 23:40] [AGENT-B] DONE: watcher.py live; drop a jpg in
  shared/requests/ and read shared/results/. Schema unchanged.
```

READ CADENCE

Re-read `roomio.txt` at the *start of every work cycle* (event-based, not clock-based). Treat new entries from Agent A or the human as high-priority. If A has requested a schema change or flagged a conflict, resolve that before continuing your own plan.

8. Functional Requirements & Acceptance

ID	Requirement	Stage
D-1	Pipeline accepts a room photo path and returns schema-valid JSON.	Orchestration
D-2	Stage 1 detects furniture regions with bounding boxes.	Detect
D-3	Stage 2 maps each region to a valid <code>archetype_id</code> from <code>archetypes.json</code> + confidence.	Classify
D-4	Unknown/low-confidence items fall back to <code>placeholder_box</code> , never crash.	Classify
D-5	Stage 3 returns a dominant color (hex + name) per region.	Color
D-6	Runs fully locally via Ollama; no paid/cloud calls.	Runtime
D-7	<code>watcher.py</code> processes <code>shared/requests/</code> → <code>shared/results/</code> with atomic writes.	Coordination
D-8	Every decision/contract change/dependency logged to <code>roomio.txt</code> with timestamp + [AGENT-B].	Comms
D-9	Never writes outside <code>/detection-pipeline</code> and <code>/shared</code> ; never edits front-end.	Isolation
D-10	A README documents how to run the watcher and the schema.	Docs

Acceptance test (the "done" bar)

Drop one of the IKEA/room reference photos into `shared/requests/` . Within a few seconds a `result.json` appears in `shared/results/` that (1) validates against the schema, (2) lists each visible furniture piece mapped to a real archetype id, (3) includes a sensible color per item, and (4) degrades gracefully (`placeholder_box`) on anything it can't confidently classify. Agent A can render confirmation dropdowns directly from it.

9. Build Milestones

Milestone	Deliverable	Definition of done
B0	Scaffold <code>/detection-pipeline</code> + <code>/shared</code> ; publish schema v1.0; log intent to <code>roomio.txt</code>	Folders exist; schema + archetypes stub committed; Agent A pinged.
B1	Ollama VLM call working on one photo (moondream2 to validate, then qwen2.5vl)	A photo yields parsed JSON of detected items.
B2	Closed-set classifier mapping to archetype ids + confidence + fallback	Every item resolves to a valid id or <code>placeholder_box</code> .
B3	k-means color stage per region	Each item gets a hex + named color.
B4	<code>pipeline.py</code> end-to-end + schema-valid output	One call: photo → full proposal JSON.
B5	<code>watcher.py</code> + atomic file handoff	requests/ → results/ loop works; Agent A can consume.
B6	Optional YOLO/HomeObjects-3K fast path; README; tests	Deterministic detector available; docs complete.

One-line summary. Build a local Ollama-based (Qwen2.5-VL) + OpenCV pipeline in `/detection-pipeline` that turns a room photo into schema-valid furniture proposals (archetype + confidence + color), hands them to the front-end via `/shared` files, and logs all intent and cross-agent requests to `roomio.txt` . Suggestion-only, fully local, isolated from Agent A.

End of brief · Roomio Furniture Detection Pipeline (Agent B) · Verify all model/dataset licenses before commercial use.